

Inductive Meta-Path Learning for Schema-Complex Heterogeneous Information Networks

Shixuan Liu[✉], Changjun Fan[✉], Kewei Cheng[✉], Yunfei Wang, Peng Cui[✉], *Senior Member, IEEE*,
Yizhou Sun[✉], *Member, IEEE*, and Zhong Liu[✉]

Abstract—Heterogeneous Information Networks (HINs) are information networks with multiple types of nodes and edges. The concept of meta-path, i.e., a sequence of entity types and relation types connecting two entities, is proposed to provide the meta-level explainable semantics for various HIN tasks. Traditionally, meta-paths are primarily used for schema-simple HINs, e.g., bibliographic networks with only a few entity types, where meta-paths are often enumerated with domain knowledge. However, the adoption of meta-paths for schema-complex HINs, such as knowledge bases (KBs) with hundreds of entity and relation types, has been limited due to the computational complexity associated with meta-path enumeration. Additionally, effectively assessing meta-paths requires enumerating relevant path instances, which adds further complexity to the meta-path learning process. To address these challenges, we propose SchemaWalk, an inductive meta-path learning framework for schema-complex HINs. We represent meta-paths with schema-level representations to support the learning of the scores of meta-paths for varying relations, mitigating the need of exhaustive path instance enumeration for each relation. Further, we design a reinforcement-learning based path-finding agent, which directly navigates the network schema (i.e., schema graph) to learn policies for establishing meta-paths with high coverage and confidence for multiple relations. Extensive experiments on real data sets demonstrate the effectiveness of our proposed paradigm.

Index Terms—Heterogeneous information networks, inductive meta-path learning, meta-paths discovery.

Manuscript received 7 July 2023; revised 31 January 2024; accepted 22 July 2024. Date of publication 29 July 2024; date of current version 5 November 2024. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62206303 and Grant 62001495, in part by China Postdoctoral Science Foundation under Grant 48919, and in part by the Science and Technology Innovation Program of Hunan Province under Grant 2023RC3009. Recommended for acceptance by M. Sugiyama. (Shixuan Liu, Changjun Fan, and Kewei Cheng contributed equally to this work.) (Corresponding authors: Changjun Fan; Kewei Cheng; Zhong Liu.)

Shixuan Liu, Changjun Fan, and Zhong Liu are with the Laboratory for Big Data and Decision, College of Systems Engineering, National University of Defense Technology, Changsha, Hunan 410073, China (e-mail: liushixuan@nudt.edu.cn; fanchangjun@nudt.edu.cn; liuzhong@nudt.edu.cn).

Yunfei Wang is with the National Key Laboratory of Information Systems Engineering, College of Systems Engineering, National University of Defense Technology, Changsha, Hunan 410073, China (e-mail: wangyunfei@nudt.edu.cn).

Kewei Cheng and Yizhou Sun are with the University of California, Los Angeles, CA 90095 USA (e-mail: viviancheng@cs.ucla.edu; yzsun@cs.ucla.edu).

Peng Cui is with the Department of Computer Science and Technology, Tsinghua University, Beijing 100190, China (e-mail: cuip@tsinghua.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TPAMI.2024.3435055>, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2024.3435055

I. INTRODUCTION

HETEROGENEOUS information networks (HINs), such as DBPedia [1], Amazon product graph [2] and protein data bank [3], have grown rapidly recently and provide remarkably valuable resources for many real-world applications, e.g. citation network analysis [4], recommendation systems [5] and drug-target interaction discovery [6]. Fig. 1 provides a toy example of an HIN, which contains multiple types of nodes (e.g., Person and City) and edges (e.g., *BornIn* and *LocatedIn*). The different types of nodes and edges entail different meanings and semantic relationships. To better describe the complex structure of HINs, we represent the two views of an HIN simultaneously as: 1) a *schema graph*, which uses entity types and relations as nodes and edges to provide the meta-level description of the network, and 2) an *instance graph*, which contains the instance-level observations of specific entities.

Meta-paths are essential in analyzing HINs as they provide a higher-level description of the network structure that captures the relationships between different types of nodes and edges in the HINs. Given two entities, a meta-path refers to a sequence of entity types and relations that exist between them. For example, in Fig. 1, the entity pair (*Max Planck*, *Germany*) is connected via following meta-paths:

$$\begin{aligned} \text{Person} &\xrightarrow{\text{BornIn}} \text{City} \xrightarrow{\text{LocatedIn}} \text{Country} \\ \text{Person} &\xrightarrow{\text{GraduateFrom}} \text{University} \xrightarrow{\text{LocatedIn}} \text{Country} \\ \text{Scientist} &\xrightarrow{\text{WorksAt}} \text{University} \xrightarrow{\text{LocatedIn}} \text{Country} \end{aligned}$$

Collectively, these meta-paths indicate that *Max Planck* is a citizen of *Germany* and provide valuable insights into answering the query (*Max Planck*, *isCitizenOf*, ?). As meta-paths are effective in encapsulating interpretable semantics for relationships and transferring knowledge for reasoning about unseen entities [7], significant efforts have been made to leverage meta-paths for HIN link prediction tasks [8], [9].

Notwithstanding the benefits of meta-paths, their applications in HINs are currently impeded by the challenges involved in their discovery. There are two main factors that pose challenges to the discovery of meta-paths in schema-complex HINs. The first entails the vast search space of potential meta-paths, and the second involves the burdensome task of accurately evaluating them through enumeration of path instances.

First, the meta-path search space expands exponentially with the meta-path length. This expansion further occurs as the number of entity types and relations in the HIN increases. Therefore, most existing meta-path-based approaches are only

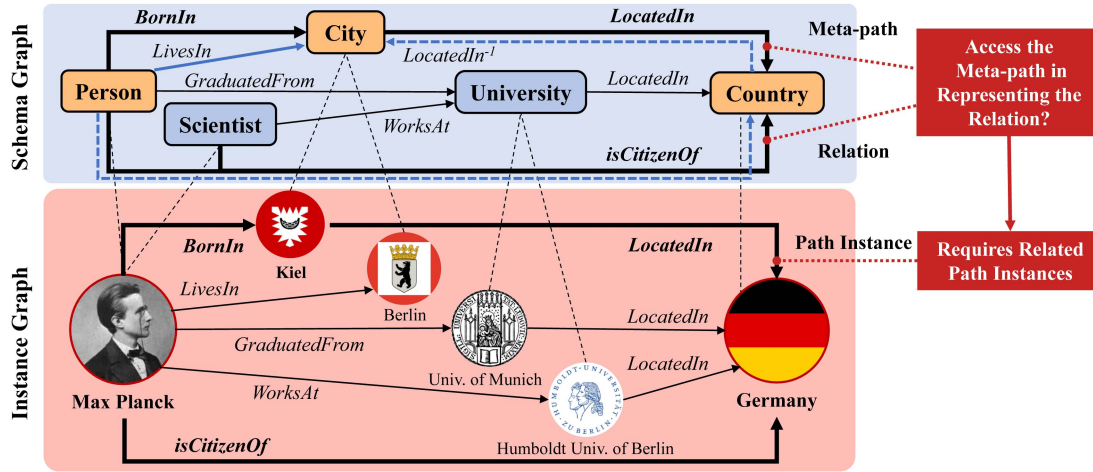


Fig. 1. An illustration of a two-view heterogeneous information network. **Top**: a schema graph $\mathcal{G}_S$ where nodes represent entity types. **Bottom**: an instance graph $\mathcal{G}_I$ where nodes represent specific entities. The entities in $\mathcal{G}_I$ are linked to one or multiple entity types in $\mathcal{G}_S$. A path in the $\mathcal{G}_S$ (one of which is boldfaced in black) is referred to as a meta-path. To learn these meta-paths, evidence from corresponding path instances, marked in black bold line in $\mathcal{G}_I$, is typically necessary.

capable of handling schema-simple HINs. These are HINs with a limited number of entity types and relations (e.g., DBLP with 4 entity types [10]), where enumeration or manual design is feasible [6], [11], [12]. Only a few works could reason schema-complex HINs such as knowledge bases (KBs) that cover a large number of entity types and relations (e.g., Dbpedia contains 174 entity types and 305 relations). This intractable nature of meta-path discovery hampers the reasoning process, leading to their under-utilization in KB reasoning, especially when dealing with queries that involve a large number of relations.

Second, meta-paths are schema-level concepts, whose plausibility is evaluated based on the observation of paths at the instance level. For instance, as depicted in Fig. 1, to identify the meta-path $\text{Person} \xrightarrow{\text{BornIn}} \text{City} \xrightarrow{\text{LocatedIn}} \text{Country}$ in explaining isCitizenOf , it is typically essential to refer to the evidence from corresponding path instances between isCitizenOf -related entity pairs, such as $\text{Max Planck} \xrightarrow{\text{BornIn}} \text{Kiel} \xrightarrow{\text{LocatedIn}} \text{Germany}$. Sampling path instances of meta-paths could not guarantee effective evaluation whereas enumerating them is time-consuming. Besides, as current methods heavily depend on path instances to discover meta-paths, they are incapable of handling inductive settings where the instance-level paths for a relation are unobserved. For example, existing methods cannot learn meta-paths to infer the query related to LivesIn in Fig. 1 as there are no observed path instances for this relation.

In this paper, we address aforementioned issues respectively by reinforcement learning (RL) and representation learning. The first issue of exploring the expansive meta-path space is tackled using RL, which offers an effective and flexible solution that can accommodate multiple training objectives for learning the meta-path discovery component. To achieve this, we parameterize the discovery component as an RL-based path-finding agent provided after the evaluation of each meta-path. This agent learns to select plausible meta-paths by listening to rewards provided after meta-path evaluation.

For the meta-path evaluation issue, we directly represent meta-paths at the schema level by learning embeddings of entity

types and relations [13]. The plausibility of meta-paths can be subsequently assessed using the learned embeddings of entity types and relations. Embeddings have a strong ability to capture the underlying similarities among entity types and relations for knowledge transfer, making them well-suited for addressing changes in inductive settings. For instance, even if we have not observed any path instances related to the relation LivesIn in Fig. 1, we can still infer the meta-path $\text{Person} \xrightarrow{\text{isCitizenOf}} \text{Country} \xrightarrow{\text{LocatedIn}^{-1}} \text{City}$ for relation LivesIn . This inference is possible due to the similarity between the meta-paths explaining BornIn (which has observed path instances) and LivesIn , captured through the embeddings.

We propose a novel method - SchemaWalk, which can explore the large search space efficiently to perform a challenging task - *inductive meta-path learning* for the first time. In contrast to the instance-level inductive setting (a typical KB task) that aims to reason unseen entities, inductive meta-path learning requires learning meta-paths without finding the evidence of relation-specific path instances.

The main contributions of this paper are as follows:

- 1) We make the first attempts to formalize an interesting problem - inductive meta-path learning and proposed a novel framework to deal with complex HINs in the inductive setting.
- 2) We frame the meta-path learning problem as a Markov Decision Process (MDP) on the schema graph and design a novel RL-based agent, which jointly refines its policy and the schema-level representations to efficiently learn meta-paths of high coverage and confidence. During inference, the trained agent could promptly output/infer meta-paths for multiple relations with/without specific evidence of path instances.
- 3) We conduct extensive experiments, and demonstrate the superior ability of SchemaWalk in learning high-quality meta-paths for both KBs and general HINs. Impressively, SchemaWalk could effectively reason relations without finding specific evidences, with comparable

performance to cases when these evidence for relations are provided.

We discuss related work systematically in Section II. In Section III, after giving the necessary definitions and learning objectives, we formulate the MDP for SchemaWalk, introduce the agent design, and outline the training pipeline. Section IV presents the results in various settings, including multi-relation inductive setting, multi-relation transductive setting and per-relation transductive setting. To validate SchemaWalk, we conduct query-answering and link prediction experiments, as well as entity-level inductive studies. We also compare SchemaWalk against state-of-the-art baseline methods using four real-world HINs. We further analyze the convergence properties and the mined meta-paths in Section V and conclude the paper in Section VI.

II. RELATED WORK

Meta-Path Reasoning: After Sun et al. proposed meta-paths to carry semantic information and gauge entity relevance [4], an increasing number of papers validated its applicability and performance in various tasks, including link prediction [8], [14]. The most pivotal step for meta-path reasoning based approaches is to discover meta-paths.

For schema-simple HINs, the simplest way is through exhaustive enumeration up to a given length [15], but is hideously computationally-expensive for schema-complex HINs. Some graph-traversal methods conduct breadth-first search [16] or A^* algorithm [17] on the network schema to generate meta-paths, however, the learning is hindered by the lacking of appropriate signal from the instance level. In particular, *Autopath* combines deep content embedding and continuous RL to learn implicit meta-paths, and calculates the similarity scores as the empirical probabilities of reaching the target entities [18].

For schema-complex HINs, the majority of current approaches involve a two-phase process that includes the generation and summarization of path instances. Lao and Cohen utilize random walks to generate meta-paths within fixed length l and describe a learnable proximity measure with the discovered meta-paths, but l is critical to the performance and varies greatly among datasets. *FSPG* is a greedy approach that could consider user-input to derive the most relevant subset of meta-paths regarding the selected entity pairs [19]. Wan et al. present *MPDRL*, an RL method that considers type context to generate path instances, as an extension to *MINERVA* [20]. However, *MPDRL* could be easily burdened by its path-finding component as its overall performance depends on partial observation (i.e., generated path instances).

Multi-Hop Reasoning: Multi-hop reasoning methods leverage the information stored in the path instances connecting entity pairs [21]. These methods mainly disregard relation types and operate on inference paths obtained by random walks [22], [23] or heuristics [24], [25]. Inspired by the advancement of deep RL, Xiong et al. first formulate the path-finding process as sequential decision making and describe an RL framework for learning multi-hop relational paths [26]. Based on this, various papers explore different RL formulations and leverage different

learning methods to boost multi-hop reasoning for prediction [20], [27].

Embedding-Based Reasoning: Many KB completion tasks focus on learning vector representations for entities and relations with existing triples before using them for predictions. Most of these approaches ignore entity types and use different space for projection, e.g. *TransE* [28], *KG2E* [29] and *RotatE* [30]. However, these methods cannot capture compositional reasoning patterns and lack explainability [23]. Recent works have also explored the embedding of HINs, premised on discovered meta-paths. Dong et al. propose *Metapath2Vec* [31] to embed HINs with crafted meta-paths, but its performance is restricted by limited existing meta-paths. In particular, *HIN2Vec* [11] uses a multi-task learning approach to jointly embed the information of different relations and instance graph structure.

III. METHODOLOGY

In this paper, we seek to learn meta-paths with high coverage and confidence. We thus begin this section with an introduction to the definitions of HIN, meta-paths, and prevalent evaluation metrics, namely coverage and confidence, for assessing meta-path plausibility. To enable SchemaWalk to perform inductive meta-path learning, we place the path-finding agent on the schema graph with trainable schema-level representations. Therefore, following the definitions, we introduce SchemaWalk in detail by focusing on the MDP formulation for meta-path learning and the agent design. Finally, we present the training algorithm.

A. Definitions and Objective

In this subsection, we provide all necessary definitions before illustrating them with a guiding example. We then formulate the learning objective.

Heterogeneous Information Network (HIN), denoted as $\mathcal{H} = (\mathcal{G}_I, \mathcal{G}_s)$ is a directed graph that encapsulates two perspectives: (1) a *schema graph* for meta-level abstraction, denoted as $\mathcal{G}_s$, and (2) an *instance graph* for instance-level instantiations, denoted as $\mathcal{G}_I$. In particular, $\mathcal{G}_I = (V, E)$ is formed with the set of entities denoted as V , and the set of links connecting entities denoted as $E \subseteq V \times V$. The type mapping $\tau : V \rightarrow 2^T$ maps an entity to entity types, where T denotes the entity type space; the relation mapping $\phi : E \rightarrow 2^R$ labels a link to several relations, where R is the relation set. The schema graph for HIN $\mathcal{H}$ is a directed graph $\mathcal{G}_s = (T, R)$ defined over entity types T , with links as sets of relations from R .

A **meta-path** is a path defined on the schema graph of a HIN. It serves as a vital tool in HIN analysis, capturing the semantic relationships between different types of entities in the network. A meta-path M of length l is defined as $M = t_1 \xrightarrow{r_1} t_2 \xrightarrow{r_2} \dots \xrightarrow{r_{l-1}} t_l$, where $t_i \in T$ denotes an entity type and $r_i \in R$ represents a relation. An instance-level path $P = v_1 \xrightarrow{r_1} v_2 \xrightarrow{r_2} \dots \xrightarrow{r_{l-1}} v_l$ satisfies M if $\forall i \in \{1, \dots, l\}, t_i \in \tau(v_i)$ and $\forall i \in \{1, \dots, l-1\}, r_i \in \phi(v_i, v_{i+1})$. In this case, we also say P is a path instance specific for M . The usage of meta-path instances is widespread for assessing the plausibility of meta-paths. In the field of association rule mining, coverage and

confidence are two of the most crucial metrics for rule evaluation, each offering a distinct perspective [32]. Coverage denotes the rule's applicability, while confidence serves as a gauge for its reliability. Here, we delve deeper these concepts and establish the rationale for combining these two metrics.

The **coverage** of a meta-path M for a specific relation r_q quantifies the frequency at which the meta-path occurs in r_q -related entity pairs. It is calculated by taking the ratio of entity pairs connected by both r_q and M , to the total number of r_q -connected entity pairs,

$$Cvr_g^{\mathcal{H}}_{M \Rightarrow r_q} = \frac{|\{(v_i, v_j) | Con_M(v_i, v_j), r_q \in \phi(v_i, v_j), v_i, v_j \in \mathcal{H}\}|}{|\{(v_i, v_j) | r_q \in \phi(v_i, v_j), v_i, v_j \in \mathcal{H}\}|} \quad (1)$$

where $Con_M(v_i, v_j)$ indicates a meta-path instance that connects the entity pair (v_i, v_j) .

The **confidence** of a meta-path M in accurately identifying r_q evaluates the trustworthiness of the associations observed along the meta-path to provide an explanation for r_q . This metric is computed as the conditional probability of observing the consequent entities in $\mathcal{G}_I$ given the presence of the antecedent entities along the meta-path M , as given below. The denominator signifies the total number of entity pairs connected by the meta-path M :

$$Conf^{\mathcal{H}}_{M \Rightarrow r_q} = \frac{|\{(v_i, v_j) | Con_M(v_i, v_j), r_q \in \phi(v_i, v_j), v_i, v_j \in \mathcal{H}\}|}{|\{(v_i, v_j) | Con_M(v_i, v_j), v_i, v_j \in \mathcal{H}\}|} \quad (2)$$

Given the entity pairs connected by a relation, coverage identifies the frequency of a meta-path being satisfied by the paths connecting these entity pairs whilst confidence measures the correctness of a meta-path's representation of the relation. Meta-paths could display high accuracy but limited applicability (high confidence, low coverage), or broad applicability but low accuracy (high coverage, low confidence). By integrating these two metrics, we could identify rules that are of greater relevance and utility. The complexity in computing these metrics for a meta-path involves finding all its path instances (also known as the evidence for it).

Example: Fig. 1 shows a toy example of HIN, with $T = \{Person, Scientist, University, City, Country\}$ and $R = \{isCitizenOf, GraduatedFrom, WorksAt, LocatedIn, BornIn, LivesIn\}$. The relations between *Max Planck* and *Germany* is $\phi(MaxPlanck, Germany) = \{isCitizen\}$ and *Max Planck* is mapped into types: $\tau(MaxPlanck) = \{Person, Scientist\}$. To reason the relation *isCitizenOf*, we could refer to following meta-path: *Person* $\xrightarrow{GraduatedFrom}$ *University* $\xrightarrow{LocatedIn}$ *Country* which is satisfied by the instance path *MaxPlanck* $\xrightarrow{GraduatedFrom}$ *Univ.ofMunich* $\xrightarrow{LocatedIn}$ *Germany*. The entity pair $(MaxPlanck, Germany)$ is thereby connected via such meta-path. If the HIN indicates following information: 250 people graduated from German universities, among them 150 are German citizens and 100 are international students, and there are 200 German citizens in total. In this sense, the coverage

of the above meta-path with respect to relation *isCitizenOf* is $\frac{150}{200}$, and the confidence is $\frac{150}{200+100}$.

Learning Objective: In this paper, we aim to learn meta-paths with high coverage and confidence for relations directly at the schema level. To achieve this goal, we train a path-finding agent on the schema graph to effectively navigate and explore the most promising meta-paths that lead to the relations. The evaluation of these meta-paths is performed using meta-path instances found in the instance graph. To demonstrate the effectiveness of our proposed methods across various scenarios, we assess the performance of our learned model using three distinct settings, as illustrated in Fig. 3. In the *multi-relation inductive setting* (Fig. 3(a)), a single model is trained to discover meta-paths for multiple relations. The relations used in training are different from the test dataset, ensuring a disjoint relationship between them. Consequently, the meta-path instances for the relations in the test dataset are unseen during the training phase. Due to the inherent challenges in the multi-relation inductive setting, many existing methods struggle to effectively address it. To comprehensively evaluate our proposed methods, we include two additional settings that are comparatively less challenging. In the *multi-relation transductive setting* (Fig. 3(b)), a single model is trained to discover meta-paths for multiple relations. The target relations used for training the model encompass the relations present in the test dataset. In the *per-relation transductive setting* (Fig. 3(c)), a separate model is trained for each single relation to discover specific meta-paths exclusively tailored for that particular relation.

B. Reinforcement Learning Formulation

As discussed in Section II, current meta-path discovery methods adopt a bottom-top design. For example, MPDRL employs RL to navigate the instance graph, creating path instances prior to summarizing them as meta-paths. This approach not only faces challenges associated with the introduction of bias through path instances sampling but also prevents the direct learning of rules at the schema level. To fully unleash the expressive power of schema-level representations, we formulate the meta-path generator as a path-finding agent on the schema graph. Given the extensive meta-path space, we adopt an RL-based agent for efficient exploration. As depicted in Fig. 2 (Left), each time for the relation r_q , the agent randomly starts at a source type t_{src} in the entity type pairs (t_{src}, t_{tgt}) connected by r_q on the $\mathcal{G}_S$. This forms the query $r_q(t_{src}, t_{tgt}) = ?$ for the agent who dynamically adjusts its policy to arrive at t_{tgt} to form meta-paths and simultaneously maximizing the rewards for the discovered meta-paths. We thus describe the meta-path learning problem, as a Markov Decision Process (MDP) on the schema graph by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$.

States: Intuitively, we wish the agent to have sufficient knowledge of its current position and all information about the query $(r_q(t_{src}, t_{tgt}) = ?)$ during its navigation. Therefore, at step i , the state S_i is represented by $(t_i, t_{src}, r_q, t_{tgt})$, where t_i stands for the current entity type node. The state space $\mathcal{S}$ contains all valid combinations in $T \times T \times R \times T$.

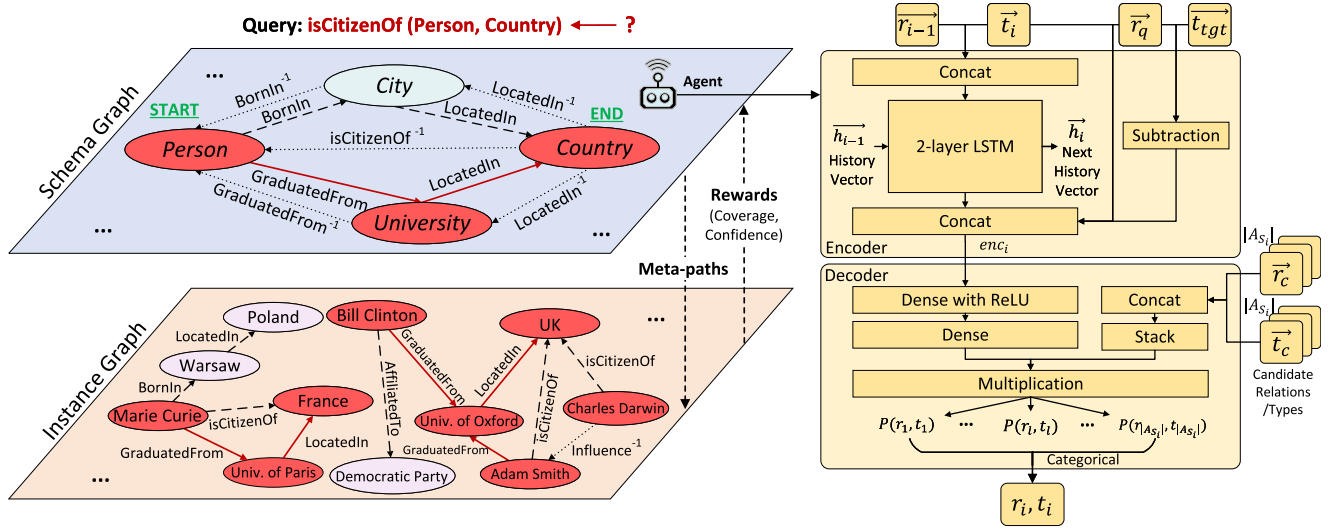


Fig. 2. Overview of SchemaWalk. The **Left** part describes the interactions between the two views of HIN. The upper schema graph provides the learning environment where the agent is trained to navigate to the target type node based on a query, e.g. $isCitizenOf(Person, Country) = ?$, and establishes meta-paths, e.g. $Person \xrightarrow{GraduatedFrom} University \xrightarrow{LocatedIn} Country$. The lower instance graph provides the rewards: the coverage and confidence of the discovered meta-path are calculated based on the instance paths satisfying the meta-path, e.g. $MarieCurie \xrightarrow{GraduatedFrom} Univ.ofParis \xrightarrow{LocatedIn} France$. Subsequently, the rewards would be utilized to train the agent. Note that the red arrows denote the discovered meta-path and related instance-paths, dashed arrows denote existing relations, and dotted arrows indicate the inverse ones. The **Right** part shows the detailed encoder-decoder based policy network architecture.

Actions: Given a state $S_i = (t_i, t_{src}, r_q, t_{tgt})$, the action space A_{S_i} contains all outgoing edges of the type node t_i in schema graph T_G along with an operation to stay unmoved, i.e. $A_{S_i} = \{(r, t) | (t_i, r, t) \in T_G\} \cup \{(\emptyset, t_i)\}$. Starting at node t_{src} , the agent selects the most favoured action for $l - 1$ times based on the representations of edge r and outgoing node t , forming a l -length meta-path or halting at t_{tgt} if it reaches the target node t_{tgt} before the maximum hop.

Transition: The environment dynamics is characterized by the state transition probability $\mathcal{P} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$. Given $S = (t_i, t_{src}, r_q, t_{tgt})$ and $A = (t_i, r, t)$, then $\mathcal{P}(S, A) = (t, t_{src}, r_q, t_{tgt})$, implying that the state is updated to the new type node t via the chosen r .

Rewards: As meta-paths are a concept at the schema level, evaluating their performance necessitates incorporating evidence from instance graphs. To achieve this, we integrate the commonly used metrics of coverage and confidence, based on meta-path instances, into the reward function. Moreover, we introduce an arrival indicator $\mathbb{I}_{arr}(M) \in \{0, 1\}$ to guide the navigation process, as outlined below:

$$\mathbb{I}_{arr}(M) = \mathbb{I}\{t_l = t_{tgt}\} \& \mathbb{I}\{\exists i \in \{1, \dots, l-1\}, A_i \neq (\emptyset, t_i)\} \quad (3)$$

where $\&$ denotes the logical-and operation. The arrival indicator ensures the agent reaches the target type node t_{tgt} (first term) and prevents the agent from sticking at the starting node t_{src} (second term). The second term is useful when the agent observes queries with the same source and target type node, such as $PlaysAgainst(Team, Team) = ?$. Otherwise, the agent may mistakenly believe that it has reached the target type ($Team$) as it starts and therefore never move, rendering no meta-paths.

The three parts are combined with different weights, resulting in normalized rewards in the range $[0, 1]$ as follows:

$$R(M|r_q, \mathcal{H}) = \frac{\lambda_1 * Cvr_{M \Rightarrow r_q}^{\mathcal{H}} + \lambda_2 * Conf_{M \Rightarrow r_q}^{\mathcal{H}} + \mathbb{I}_{arr}(M)}{\lambda_1 + \lambda_2 + 1} \quad (4)$$

C. Agent Design

With the above formulation, the agent learns to navigate the schema graph by listening to the reward signal. However, such formulation alone could not support meta-path learning in the absence of specific evidence. Consequently, we incorporate representation learning to further empower the search ability and, more importantly, encapsulate the underlying similarities among entity types and relations. The latter is particularly useful when the agent learns to navigate based on an unobserved query, as in inductive meta-path learning settings. Specifically, we design the policy network with an encoder-decoder architecture that incorporates schema-level representations. In this section, we denote vector representations with bold fonts.

Policy Network: As illustrated in Fig. 2 (Right), the policy network adopts an encoder-decoder architecture. The encoder is parameterized by a two-layer long short term memory network (LSTM), which encodes the state S_i at each time step i into a vector representation $\mathbf{S}_i$. While other models like Transformers could have been considered, we opted for LSTM due to its fewer parameters, enabling rapid convergence while yielding satisfactory performance, even with limited sequence data. Specifically, the LSTM takes the concatenation of the vector representations of relation $\mathbf{r}_{i-1} \in \mathbb{R}^{d_e}$ and type node $\mathbf{t}_i \in \mathbb{R}^{d_e}$ as inputs and outputs $\mathbf{S}_i$ and the updated history vector $\mathbf{h}_i$. To

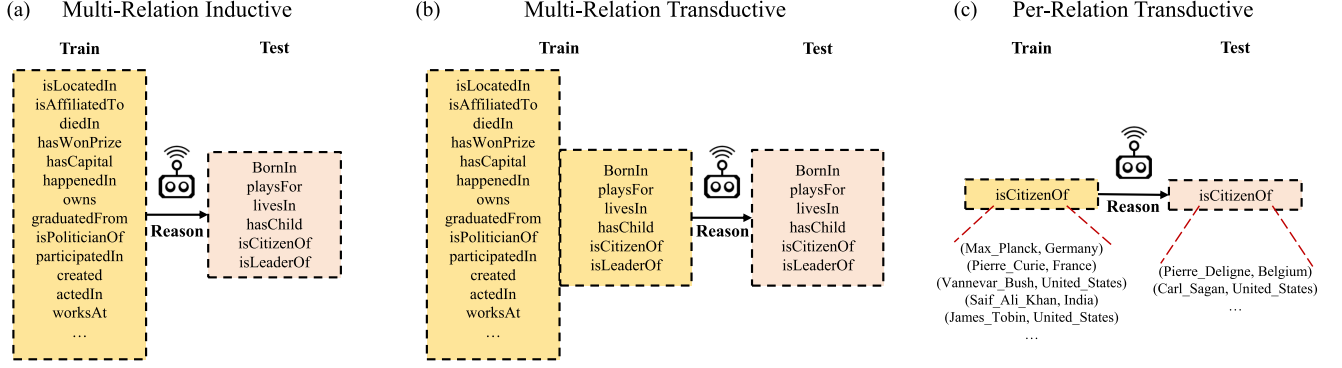


Fig. 3. Three experimental settings adopted in this paper (exemplified using the relations and entity pairs in YAGO26K-906).

form the encoding enc_i , $\mathbf{S}_i$ is concatenated with $\mathbf{t}_i$, $\mathbf{r}_q$, and the difference between $\mathbf{t}_{\text{tgt}}$ and $\mathbf{r}_q$. The design of the difference item helps the agent find the adjacent head types for the query's target type. We use $[\cdot \parallel \cdot]$ to represent the concatenation operator.

$$\begin{aligned} \mathbf{h}_i, \mathbf{S}_i &= \text{LSTM}(\mathbf{h}_{i-1}, [\mathbf{r}_{i-1} \parallel \mathbf{t}_i]) \\ \text{enc}_i &= [\mathbf{S}_i \parallel \mathbf{t}_i \parallel \mathbf{r}_q \parallel (\mathbf{t}_{\text{tgt}} - \mathbf{r}_q)] \end{aligned} \quad (5)$$

The decoder is implemented by a two-layer multi-layer perceptron (MLP) network (whose hidden size is denoted as d_h) with rectified linear unit. All the candidate relations r_c and type nodes t_c are determined by the outgoing edges of the current type node. Each edge is represented by the concatenation of $\mathbf{r}_c$ and $\mathbf{t}_c$. These representations are stacked to create a feature matrix for decision D_i . After passing the encoding enc_i to the MLP, the output is multiplied by D_i and then it goes through a softmax layer to calculate the action probability $\mathbf{P} \in \mathbb{R}^{|A_{S_i}|}$,

$$\begin{aligned} D_i &= \circ([\mathbf{r}_c \parallel \mathbf{t}_c], c = \{1, \dots, |A_{S_i}|\}) \\ \mathbf{P} &= \text{softmax}(D_i (W_2(\text{ReLU}(W_1 \text{enc}_i + b_1)) + b_2)) \end{aligned} \quad (6)$$

where, $\circ(\cdot)$ is the stacking operator. The overall policy network is represented by π_θ , where θ corresponds to the parameters in the LSTM and MLP. Based on the action probability distribution $\mathbf{P}$, the agent selects an action and subsequently moves to the next type node with regard to the transition dynamics.

D. Training Pipeline

Due to the complex nature of meta-path finding, we leverage RL algorithm to train the policy network π_θ of the agent to maximize the rewards described in (4). To support multi-relation reasoning, we employ a periodic design that involves switching the relation for training every I_{base} iterations, enabling each relation to undergo I_r rounds of training. The training objective is,

$$J(\theta|r_q, \mathcal{H}) = \mathbb{E}_{(t_{\text{src}}, r_q, t_{\text{tgt}}) \sim \mathcal{H}} \mathbb{E}_{M \sim \pi_\theta(M)} [R(M|r_q, \mathcal{H})] \quad (7)$$

where, $\pi_\theta(M)$ is the distribution for the generated l -length meta-path following policy π_θ . Following the REINFORCE algorithm [33], [34], the objective could be optimized in the

direction of,

$$\begin{aligned} \nabla_\theta J(\theta|r_q, \mathcal{H}) &= \mathbb{E}_{(t_{\text{src}}, r_q, t_{\text{tgt}}) \sim \mathcal{H}} \mathbb{E}_{M \sim \pi_\theta(M)} [R(M|r_q, \mathcal{H}) \nabla_\theta \ln \pi_\theta(M)] \\ &= \mathbb{E}_{(t_{\text{src}}, r_q, t_{\text{tgt}}) \sim \mathcal{H}} \mathbb{E}_{M \sim \pi_\theta(M)} \left[R(M|r_q, \mathcal{H}) \sum_{i=1}^l \nabla_\theta \ln \pi_\theta(A_i|S_i) \right] \end{aligned} \quad (8)$$

To estimate the gradient in (8), we randomly draw K entity type pair $(t_{\text{src}}, t_{\text{tgt}})$ connected by r_q from $\mathcal{H}$ and run N roll-outs for each sample. The gradient is approximated by the sampled trajectories as follows,

$$\begin{aligned} \nabla_\theta J(\theta|r_q, \mathcal{H}) &\approx \frac{1}{N \cdot K} \sum_{j=1}^{N \cdot K} \left[R(M|r_q, \mathcal{H}) \sum_{i=1}^l \nabla_\theta \ln \pi_\theta(A_i^j|S_i^j) \right] \end{aligned} \quad (9)$$

We append a moving average baseline to stabilize the training process by reducing the variance. The baseline is calculated by averaging the accumulative discounted rewards. Although the Actor-Critic pipeline with a parametric baseline prevails in the field of RL [35], we do not observe statistical significance by adopting it. Besides, to spur the exploratory behavior of SchemaWalk in discovering diverse meta-paths, we include an entropy regularization term with weight β (with a decay rate) in the loss function. Ultimately, the ADAM optimizer is used to minimize the loss by rate α . With above training process, the policy network π_θ guides the learning of trainable representations of entity types and relations with the goal of maximizing the reward.

IV. EXPERIMENTS

In this section, we present the empirical experimental results to establish following points: (i) SchemaWalk can effectively and efficiently generate meta-paths without relation-specific evidence (multi-relation inductive setting, as in Section IV-A1), (ii) SchemaWalk is competitive against state-of-the-art KB reasoning baselines in the query answering task (multi-relation transductive setting, as in Section IV-A2), (iii) SchemaWalk is effective in reasoning general HINs (per-relation transductive

TABLE I
CATEGORIES OF BASELINE METHODS

Names	Meta-path-based	Embedding-based	Rule-learning
RotatE		✓	
TransE		✓	
DistMult		✓	
ComplEx		✓	
Random Walk	✓		
MPDRL	✓		
PCRW	✓		
Autopath	✓		✓
Metapath2Vec	✓	✓	
HIN2Vec	✓	✓	
NHSE		✓	
RNNLogic			✓
MINERVA			✓
MLN4KB			✓

setting, as in Section IV-B). The distinctions between the three settings are illustrated in Fig. 3. In the per-relation transductive setting (which is prevalent in the current HIN reasoning experiments), the agent is trained and tested over entity pairs connected by a specific relation in each run. For the two multi-relation settings, we train and test our model on entity pairs associated with multiple relations. In the multi-relation inductive setting, the trained and test relations do not overlap. Overall, we compare against the baselines shown in Table I.

A. Multi-Relation Experiments

We evaluate the multi-relation transductive and inductive capabilities of SchemaWalk by query answering tasks [20]. Specifically, given the query $(e_h, r_q, ?)$, we aim to identify the tail entity e_t . In multi-relation inductive experiments, the agent learns the meta-path generation mechanism for the trained relations and infers rewarding meta-paths for the test relations.

Datasets: We use two large KBs: YAGO26K-906 and Dbpedia for multi-relation settings. The meta-path space for some relations in YAGO26K-906 is showcased in Table III. We can see that, even in YAGO26K-906 which has comparatively fewer entity types and relations, each relation entails a million-scale meta-path space to search over and has extremely low valid rate. Here, a meta-path is valid if it has any meta-path instance in the $\mathcal{G}_I$, and low valid rate reflects the difficulty of mining meaningful meta-paths from the meta-path space. Besides, for DB111K-174, which contains 305 relations, we train and test on sampled relations. A detailed description on the two datasets and the chosen train/test relations could be found at Appendix B, (available online).

Inference: During inference, we utilize SchemaWalk to generate a set of high-quality meta-paths to reason about each test relation by employing a beam search with a large beam width of 400. Afterwards, the confidence of each outputted meta-path with regard to the queried relation r_q is calculated, and is used as the score to infer tail entities. Given $(e_h, r_q, ?)$, we apply the mined meta-paths on e_h and rank possible tail entities after aggregating (max-pooling) the confidence of each meta-path. Entities that are inaccessible from e_h via any meta-path are given a rank of infinite value. We use the standard metrics for KB completion tasks, i.e., the Hits@1, 3, 10 and the mean reciprocal rank (MRR). All results are averaged over five independent runs.

1) Multi-Relation Inductive Experiments:

Baselines: We notice that there is no existing method in the literature that could learn rules for relations without specific evidence. To provide a fair comparison, we test SchemaWalk against random walk and breadth-first search based enumeration in this setting. We set the same search attempts for SchemaWalk and random walk and we also evaluate the effect of increasing the search attempts for random walk (to 5 times and 10 times the normal attempts).

Observations: Table II displays the multi-relation inductive results on YAGO26K-906 and DB111K-174. SchemaWalk significantly outperforms random walk in terms of efficiency and effectiveness. Even when we increase the search attempts for random walk by a factor of ten, it still cannot beat the accuracy of our approach on YAGO26K-906. On DB111K-174, SchemaWalk exhibits more significant superiority over the random walk method. Enumeration presents the best prediction answers on YAGO26K-906, but at an enormous computational cost (consuming nearly 498 times the inference time than SchemaWalk). Furthermore, enumeration cannot scale to DB111K-174 given the enormous meta-path space. Notably, SchemaWalk produces meta-paths with significantly higher valid rate compared to other methods, which is about 6.5 times the valid rate obtained using random walk, and 13 times obtained using enumeration. This indicates that SchemaWalk indeed learns the mechanism to generate meaningful and relevant meta-paths.

2) Multi-Relation Transductive Experiments:

Baselines: We compare SchemaWalk with six widely-used KB reasoning baselines: TransE [28], DistMult [36], ComplEx [37], RotatE [30], MINERVA [20], RNNLogic [38] and MLN4KB [39]. RNNLogic could hardly scale to DB111K-174 and we restrict the rule length to 3. Detailed information on their implementation could be referenced in Appendix A, (available online).

Observations: In Table IV, we show the multi-relation transductive results on YAGO26K-906 and DB111K-174. SchemaWalk demonstrates superior performance over three embedding-based methods, namely TransE, DistMult, and ComplEx, and wins a noticeable advantage over the path-based MINERVA and the rule-based MLN4KB. On YAGO26K-906, SchemaWalk outperforms all other models in the Hits@10 metric and performs comparably with the rule-learning RNNLogic across all metrics. Additionally, on DB111K-174, SchemaWalk achieves the highest scores for Hits@3, Hits@10, and the second-highest score for MRR. Overall, SchemaWalk competes strongly with state-of-the-art models for transductive KG query-answering.

Although RotatE exhibits fair competitiveness with SchemaWalk on both datasets, it is prone to being influenced by unseen entities due to its embedding-based nature. We would further elaborate this issue in Section IV-B3. Besides, RNNLogic encounters scalability issues and takes a remarkably long time of 7 days to complete even one run on DB111K-174.

3) Comparing the Multi-Relation Inductive and Multi-Relation Transductive Capacity of SchemaWalk: Fig. 4 shows the performance drop of SchemaWalk from multi-relation

TABLE II

INDUCTIVE KG REASONING RESULTS ON YAGO26K-906 AND DB111K-174, AVERAGED OVER 5 RUNS. ENUMERATION CANNOT SCALE TO DB111K-174 AND HENCE THE RESULTS ARE NOT REPORTED

		SchemaWalk	Random Walk	Random Walk (*5)	Random Walk (*10)	Enumeration
YAGO26K-906	Hits@1	<u>0.146</u>	0.066	0.114	0.121	0.312
	Hits@3	<u>0.239</u>	0.097	0.169	0.191	0.452
	Hits@10	<u>0.376</u>	0.139	0.220	0.288	0.564
	MRR	<u>0.218</u>	0.089	0.151	0.174	0.400
DB111K-174	Hits@1	0.638	0.010	0.032	0.070	-
	Hits@3	0.825	0.014	0.048	0.105	-
	Hits@10	0.851	0.017	0.062	0.137	-
	MRR	0.731	0.012	0.043	0.093	-

		SchemaWalk	Random Walk	Random Walk (*5)	Random Walk (*10)	Enumeration
YAGO26K-906	#Output Metapaths	20,326	19,037	92,003	187,400	14,894,588
	#Valid Metapaths	2,687	387	1,561	2,940	152,502
	Valid Rate	13.22%	2.03%	1.70%	1.57%	1.02%
DB111K-174	#Output Metapaths	30,847	22,506	111,892	222,143	-
	#Valid Metapaths	979	111	448	788	-
	Valid Rate	3.17%	0.49%	0.40%	0.35%	-

The best/second best metrics are bolded/underlined.

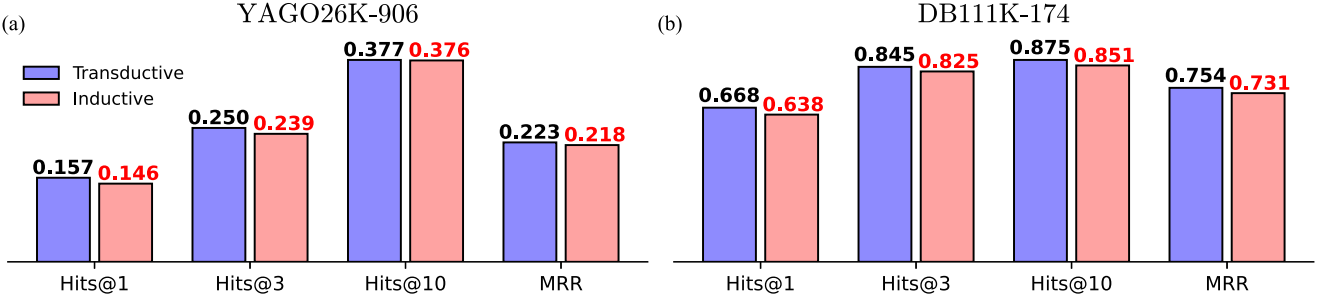


Fig. 4. The performance difference of SchemaWalk between multi-relation transductive setting and multi-relation inductive setting.

TABLE III
META-PATH SPACE FOR SOME RELATIONS IN YAGO26K-906

	#Possible Metapaths	#Valid Metapaths	Valid Rate
wasBornIn	4, 890, 928	59, 520	1.22%
LivesIn	3, 937, 737	29, 751	0.76%
hasChild	2, 172, 857	12, 469	0.57%
isCitizenOf	1, 810, 611	39, 778	2.20%
PlaysFor	1, 075, 622	1, 143	0.14%
isLeaderOf	1, 006, 833	9, 511	0.94%

inductive experiments to multi-relation transductive ones. Impressively, the performance degradation is negligible across all metrics, with an average decline rate of merely 3.32%. This observation suggests that SchemaWalk can effectively learn a policy to establish high-quality meta-paths by examining meta-path samples (and their scores) generated during training on some relations. With this policy, the model can efficiently predict meta-paths for test relations by referring to the trained schema-level representations.

B. Per-Relation Experiments

In the HIN literature [19], [40], the effectiveness of models is typically evaluated in a transductive manner and relation-wise.

Following this established practice, we demonstrate the effectiveness of SchemaWalk in reasoning over schema-complex and general HINs in this section.

Baselines: We compare SchemaWalk with eight existing relation reasoning methods: MPDRL [40], PCRW [41], Autopath [18], Metapath2Vec [31], HIN2Vec [11], NSHE [42], RotatE [30], TransE [28], MINERVA [20]. See Appendix A, available online for their detailed information.

Datasets: To validate SchemaWalk's relation-wise reasoning ability on schema-complex HINs, we run experiments on YAGO26K-906 and NELL. We also include a schema-simple HIN, Chem2Bio2RDF, to demonstrate that SchemaWalk is also useful for general HINs. The details about the chosen datasets are in Appendix B, available online. We consider three relations each for the two KBs: {isCitizenOf, DiedIn, GraduatedFrom} for YAGO26K-906 and {WorksFor, CompetesWith, PlaysAgainst} for NELL. For Chem2Bio2RDF, we focus on predicting drug-target connectivity, i.e., the relation *bind*.

Dataset Preparation: To predict relation r_q with meta-paths of maximum length l , we list all r_q -connected entity pairs and check each pair using a breadth-first search to determine whether there exists an instance path connecting them within $l - 1$ hops (apart from directly via r_q). The entity pairs that fail this check are filtered out. Unlike the practice in MPDRL [40], we do not

TABLE IV
TRANSDUCTIVE KG REASONING RESULTS ON YAGO26K-906 AND DB111K-174, AVERAGED OVER 5 RUNS

		SchemaWalk	TransE	DistMult	ComplEx	RotatE	MINERVA	RNNLogic	MLN4KB
YAGO26K-906	Hits@1	0.157	0.092	0.039	0.069	0.180	0.133	<u>0.161</u>	0.128
	Hits@3	0.250	0.225	0.052	0.123	<u>0.260</u>	0.214	0.263	0.236
	Hits@10	0.377	0.316	0.118	0.174	0.347	0.310	<u>0.364</u>	0.342
	MRR	0.223	0.176	0.059	0.106	0.237	0.192	<u>0.225</u>	0.213
DB111K-174	Hits@1	0.668	0.538	0.359	0.626	0.768	<u>0.684</u>	0.671	0.669
	Hits@3	0.845	0.809	0.481	0.728	<u>0.822</u>	0.786	0.754	0.722
	Hits@10	0.875	0.858	0.555	0.769	0.849	<u>0.863</u>	0.783	0.785
	MRR	<u>0.754</u>	0.675	0.431	0.682	0.799	0.745	0.716	0.710

The best/second best metrics are bolded/underlined.

TABLE V
PER-RELATION LINK PREDICTION RESULTS FOR YAGO26K-906, NELL AND CHEM2Bio2RDF, AVERAGED OVER 5 RUNS

		SchemaWalk	MPDRL	PCRW	Autopath	Metapath2Vec	HIN2Vec	NSHE	RotatE	TransE	MINERVA
YAGO26K-906											
isCitizenOf	ROC-AUC	0.865	0.778	0.584	0.755	0.652	0.800	0.788	0.778	0.590	<u>0.828</u>
	AP	0.904	0.793	0.706	0.723	0.781	0.837	0.803	0.830	0.810	<u>0.840</u>
DiedIn	ROC-AUC	0.947	0.710	0.645	0.723	0.661	0.785	0.748	<u>0.860</u>	0.622	0.632
	AP	0.971	0.679	0.686	0.787	0.830	0.877	0.778	<u>0.907</u>	0.745	0.786
GraduatedFrom	ROC-AUC	0.896	0.664	0.586	0.724	0.661	0.803	0.815	<u>0.830</u>	0.662	0.609
	AP	0.912	0.743	0.664	0.718	0.783	0.842	0.839	<u>0.855</u>	0.750	0.718
NELL											
WorksFor	ROC-AUC	0.913	0.759	0.646	0.703	0.613	0.790	0.774	<u>0.868</u>	0.637	0.767
	AP	0.945	0.871	0.735	0.778	0.819	0.870	0.859	<u>0.911</u>	0.719	0.802
PlaysAgainst	ROC-AUC	<u>0.907</u>	0.774	0.567	0.544	0.761	0.783	0.814	0.966	0.845	0.541
	AP	<u>0.938</u>	0.823	0.745	0.691	0.904	0.917	0.883	0.993	0.928	0.683
CompetesWith	ROC-AUC	0.706	0.589	0.547	0.567	0.600	0.730	0.677	0.870	<u>0.774</u>	0.585
	AP	0.806	0.695	0.699	0.685	0.735	0.867	0.784	0.939	<u>0.908</u>	0.702
Chem2Bio2RDF											
Bind	ROC-AUC	1.000	0.813	0.609	0.685	0.759	0.880	0.901	<u>0.936</u>	0.909	0.715
	AP	1.000	0.890	0.723	0.792	0.914	0.939	0.963	<u>0.984</u>	0.978	0.816

The bold/underlined results respectively indicate the best/second best performances for each relation among all methods.

set maximum number of attempts for the search to help include more condition-satisfying entity pairs for fair comparison. The remaining pairs form the dataset, which is split into train/test set in an 8 : 2 ratio. Before training, facts in the test set are removed from the instance graph, providing the groundings to calculate coverage and confidence.

Training Settings for SchemaWalk: On the schema graph, a relation could connect multiple entity type pairs (t_{src}, t_{tgt}). On each attempt, the agent chooses one such pair to form a query, starts at t_{src} and finds a rewarding path to t_{tgt} . For YAGO26K-906 and Chem2Bio2RDF, the query set for training is all possible entity type pairs. However, in NELL, some relations could connect an enormous number of entity type pairs (e.g., 1488 for *CompetesWith*) and many of these type pairs map to very few entity pairs. Therefore, for SchemaWalk, we narrow the query set on NELL by the minimum type pairs that cover a threshold percent (set as 80%) of the r_q -related entity pairs. See Appendix D, available online for further training settings.

Link Prediction Settings: For link prediction, the test set mentioned above constitutes the positive pairs. Following the practice in [40], we generate the negative pairs by replacing the target entity in instance graph samples with a fake one but of the same type. All negative samples for the six relations in YAGO26K-906 and NELL are generated in this manner. The

Chem2Bio2RDF already includes such negative dataset and we adopt it directly. The positive/negative rate is 2:1. To calculate similarity for each pair, we sum up the confidence of all the meta-paths that connect the pair. Ultimately, we adopt a linear regression model with L1-regularization to perform link prediction. For the four embedding-based approaches, we calculate the Hadamard product of the head and tail entity embeddings and use an SVM classifier for link prediction.

Evaluation Metrics: We use two diagnostic metrics: the area under the receiver operating characteristic curve (ROC-AUC) and average precision (AP). The results for each method were averaged over five independent runs.

1) *Results on Schema-Complex HINs:* The ROC-AUC and AP for SchemaWalk and the baselines for the link prediction task on the six selected relations are presented in Table V. SchemaWalk excels in all relations in YAGO26K-906 and the relation *WorksFor* in NELL, only lagging behind by RotatE for relations *PlaysAgainst* and *CompetesWith* in NELL.

Among the meta-path based baselines, HIN2Vec achieves decent prediction accuracy. It generates superior embeddings compared to Metapath2Vec, by utilizing meta-paths for multiple relations. The performance of NSHE is also reasonably good, similar to HIN2Vec, due to the high-order structure-derived embeddings it obtains. MPDRL produces relatively satisfactory results, thanks to its RL-based strategy to explore instance

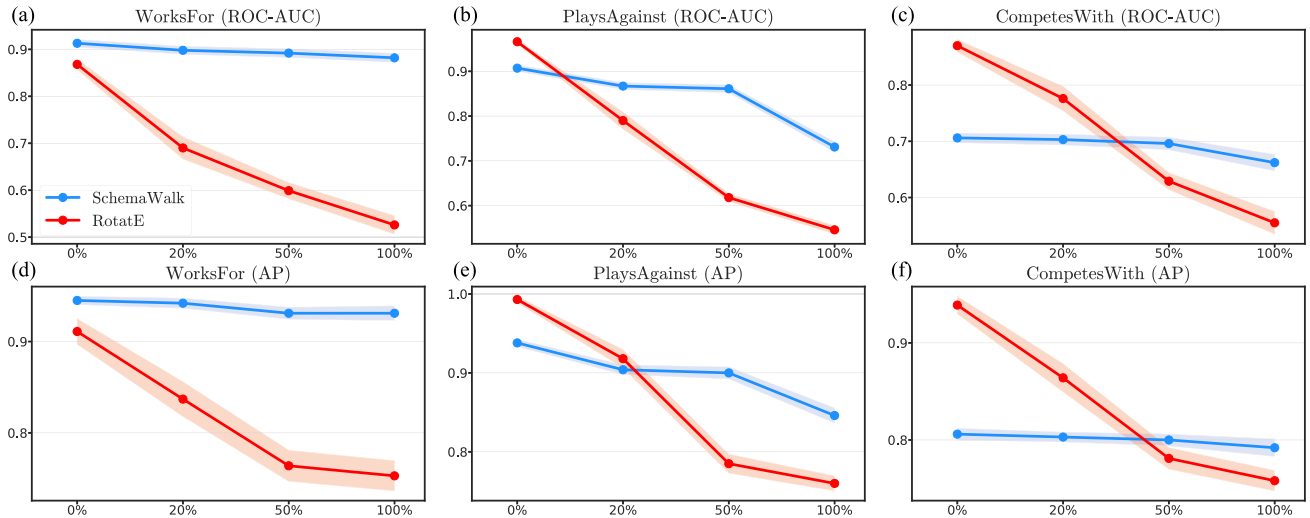


Fig. 5. Entity-level inductive experiment results for SchemaWalk (Blue) and RotatE (Red). The three columns represent the results for three relations and the above/below three charts respectively show the ROC-AUC/AP values. The horizontal axes represent the node removal rate. The shadow area represents the confidence intervals over 5 runs.

paths. Autopath exhibits acceptable capability, but its ability is confined by the limited number of discovered meta-paths, particularly for relations that involve an extensive meta-path space. PCRW is the most mediocre model among them because of its randomness-based nature. For the multi-hop-based MINERVA, it performs well in reasoning *isCitizenOf* and *WorksFor*, but performs poorly for other relations. We observe that embedding-based approach for KBs, particularly RotatE, dominates the two relations in NELL. Upon further inspection, we notice that both *PlaysAgainst* and *CompetesWith* involve thousands of type pair, suggesting an enormous meta-path space (ten million-scale) to explore.

2) *Results on Schema-Simple HIN*: We show the link prediction results on Chem2Bio2RDF in Table V. As demonstrated by the results, the meta-paths generated by SchemaWalk can distinguish all positive facts from negative facts. Besides, SchemaWalk only takes an average of 30 iterations (or roughly 400 seconds) to learn all the necessary meta-paths for link prediction. In contrast, other baselines are much more time-consuming, often taking several hours due to the gigantic instance graph of Chem2Bio2RDF. Our model is effective and efficient for general HINs.

3) *Entity-Level Inductive Experiment*: In the Entity-level inductive experiment, we concentrate on the NELL dataset and compared our method against RotatE. We randomly sample 40% of the positive testing set, remove 0%, 20%, 50% and 100% of the nodes that appear in the pairs from the instance graph, followed by regular training and link prediction. The ROC-AUC and AP results for the two methods under the four scenarios are shown in Fig. 5.

We discover that, with the removal rate of only 20%, SchemaWalk already beats RotatE for relation *PlaysAgainst* in terms of ROC-AUC and wins an all-out victory for all selected relations with the removal rate over 50%. The performance of RotatE deteriorates drastically with the removal

rate, whilst SchemaWalk only projects minor signs of being affected. We only observe relatively evident performance drop for SchemaWalk when reasoning *PlaysAgainst*, as the removal rate reaches 100% from 50% (ROC-AUC dropped by 15.1% and AP dropped by 6%). We attribute this phenomenon to the removal of some key instance nodes, which cut off massive instance paths and drastically affect the coverage and confidence score.

V. FURTHER ANALYSIS

A. Convergence Properties

We present the training curves for arrival rate, confidence and coverage on relations in YAGO26K-906 and NELL, as displayed in Fig. 7. All curves exhibit an upward trend as the training starts. Most of the curves for arrival rate and coverage continue to grow until convergence. Conversely, the confidence curves for some relations initially ascend to a high point, but converge to a sub-optimal value eventually. This happens because the agent prioritizes coverage over confidence at times, believing that finding a set of rules that covers the majority of entity pairs is more crucial than identifying only a few rules with the highest confidence.

During the initial training period, SchemaWalk exhibits noticeable exploratory behaviour, thanks to the regularization item. Nevertheless, this behaviour does not adversely affect the learning of the agent, as the confidence and coverage curves converge eventually, even for relations *PlaysAgainst* and *CompetesWith* that connect thousands of entity type pairs. The arrival rate curves tend to converge faster compared to the confidence and coverage curves, indicating that the agent needs to know how to navigate to the target type node before learning to generate high quality meta-paths. The majority of the confidence/coverage curves end up converging at different values, reflecting the variations in the confidence/coverage distribution for meta-paths amongst different relations. Lastly, all curves converge before

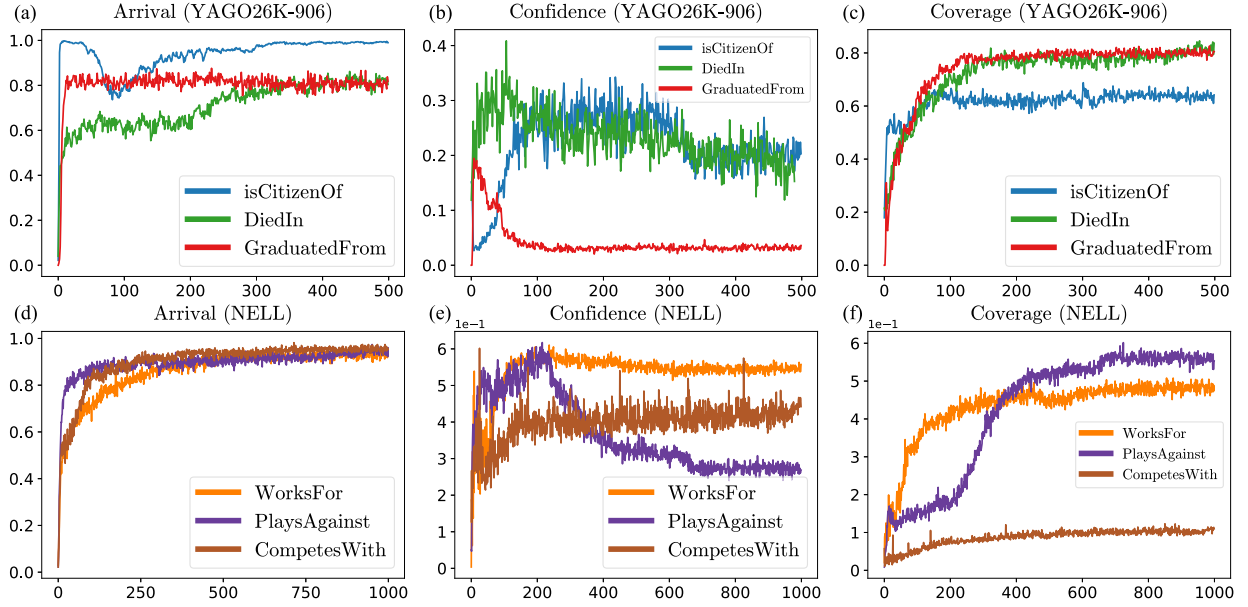


Fig. 6. SchemaWalk's training curves for arrival rate, confidence and coverage on YAGO26K-906 and NELL, averaged over 5 runs.

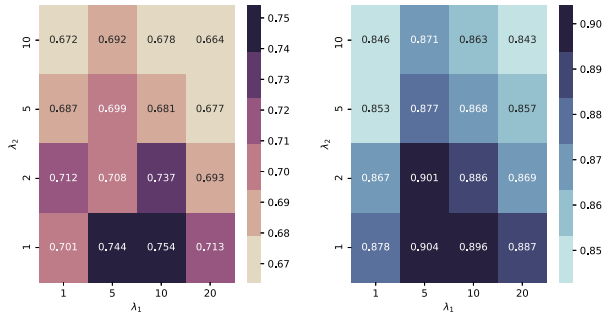


Fig. 7. The left and right plots respectively depict the results for transductive KG reasoning on DB11K-174 (measured by MRR) and per-relation link prediction on YAGO26K-906 with the relation *isCitizenOf* (expressed in AUC) with varying values for (λ_1, λ_2) .

the set number of iterations and further training iterations do not empirically yield better prediction results.

With aforementioned notations, the overall complexity for the training process is bounded by $\mathcal{O}(KN(l-1 + (|V|\Delta(\mathcal{G}_I)^2)^{l-2}))$ per epoch (See Appendix E.1, available online for detailed analysis). Even when training on the most complex schema graph, that of NELL, our method demonstrates remarkable efficiency. The observed wall times for our model to complete a single epoch for the relations *PlaysAgainst*, *CompetesWith*, and *PlaysFor* are 1.44 s, 1.98 s, and 5.01 s respectively. Consequently, it took only about 868 s, 793 s, and 1353 s respectively to converge to a reasonable point for these relations. This observation affirms that our strategy, which involves searching for the target type node within the schema graph, delivers efficient training.

B. Meta-Paths Analysis

We examine the generated meta-paths and select those meta-paths with high rewards, which are presented in Table VI. A

careful observation of these meta-paths reveals their significant diversity and contextual relevance. For instance, our model can learn multiple meta-paths to deduce citizenship via birthplace, alma mater, or workplace. Furthermore, it learns meta-paths not just for top-hierarchy types like *Person*, but also for specific types from lower hierarchies like *CEO* and *Scientist*. In terms of semantic correctness, each of these meta-paths precisely encapsulates and expresses the semantic meaning of the corresponding relations. An example includes our model learning the meta-path that infers an individual's alma mater by taking into account the employment location of their university instructor. We also observe that our model is capable of using synonymous relations to explain target relations, as demonstrated by utilizing *CompetesWith* for *PlayAgainst* and *Nationality* for *StateOfOrigin*. In multi-relation inductive settings, SchemaWalk also infer high-quality meta-paths for untrained relations (e.g., *StateOfOrigin* and *MusicalBand* in DB11K-174).

The coverage and confidence of above meta-paths are evaluated under the relevant KB. We thus observe that 16.7% of the scientists entailing *isCitizenOf* would have information of where their alma maters locate and, 54.0% of the target countries indicated by such meta-path are the true answers. We could also observe some interesting facts for politicians, scientists and coaches. For example, we find that 12.5% of the scientists would graduate from the same university where their scientist spouse works.

C. Parameter Analysis on λ_1 and λ_2

We scrutinize the impact of parameters (λ_1, λ_2) , which signify respective weights for coverage and confidence as expressed in (4), on the performance of our method. We selected values for λ_1 from the set $\{1, 5, 10, 20\}$ and for λ_2 from $\{1, 2, 5, 10\}$. The scale for these parameters is not identical due to a typically smaller value of coverage in relation to confidence. With these

TABLE VI
EXAMPLE META-PATHS OUTPUTTED/INFERRED BY SCHEMAWALK

Relations	Meta-path	Cvrg.	Conf.
WorksFor (NELL)	Person $\xrightarrow{\text{BelongsTo}}$ Company	0.505	0.818
	Person $\xrightarrow{\text{CollaboratesWith}}$ Company	0.564	0.460
	Person $\xrightarrow{\text{Controls}}$ Company	0.152	0.767
	CEO $\xrightarrow{\text{Leads}}$ Company	0.882	0.745
	Journalist $\xrightarrow{\text{WritesFor}}$ Newspaper	0.741	0.399
PlayAgainst (NELL)	Team $\xrightarrow{\text{CompetesWith}}$ Team	0.149	0.975
	Team $\xrightarrow{\text{PlaysIn}}$ League $\xrightarrow{\text{SuperPartOf}}$ Team	0.525	0.295
	Team $\xrightarrow{\text{WinTrophy}}$ Game $\xrightarrow{\text{Participant}}$ Team	0.018	0.782
	Team $\xrightarrow{\text{KnownAs}}$ Team $\xrightarrow{\text{CompetesWith}}$ Team	0.305	0.579
	Coach $\xrightarrow{\text{BelongsTo}}$ League $\xrightarrow{\text{LeagueTeam}}$ Team	0.707	0.212
isCitizenOf (YAGO26K-906)	Person $\xrightarrow{\text{BornIn}}$ District $\xrightarrow{\text{LocatedIn}}$ Country	0.295	0.083
	Person $\xrightarrow{\text{DiedIn}}$ District $\xrightarrow{\text{LocatedIn}}$ Country	0.148	0.070
	Person $\xrightarrow{\text{LivesIn}}$ Country	0.082	0.152
	Scientist $\xrightarrow{\text{GraduatedFrom}}$ University $\xrightarrow{\text{LocatedIn}}$ Country	0.167	0.540
	Scientist $\xrightarrow{\text{WorksAt}}$ University $\xrightarrow{\text{isLocatedIn}}$ Country	0.205	0.421
GraduatedFrom (YAGO26K-906)	Person $\xrightarrow{\text{isCitizenOf}}$ Country $\xrightarrow{\text{LocatedIn}^{-1}}$ University	0.169	0.034
	Person $\xrightarrow{\text{AdvisedBy}}$ Scientist $\xrightarrow{\text{WorksAt}}$ University	0.080	0.260
	Politician $\xrightarrow{\text{isPoliticianOf}}$ Country $\xrightarrow{\text{LocatedIn}^{-1}}$ University	1.000	0.133
	Scientist $\xrightarrow{\text{WorksAt}}$ University	0.268	0.191
	Scientist $\xrightarrow{\text{MarriedTo}}$ Scientist $\xrightarrow{\text{WorksAt}}$ University	0.030	0.125
StateOfOrigin (DB111K-174, Inferred)	Person $\xrightarrow{\text{Nationality}}$ Country	0.651	0.980
	Person $\xrightarrow{\text{BirthPlace}}$ Country	0.261	0.087
	Person $\xrightarrow{\text{DeathPlace}}$ Country	0.091	0.080
	Person $\xrightarrow{\text{Residence}}$ Country	0.063	0.204
	Person $\xrightarrow{\text{Predecessor}}$ Person $\xrightarrow{\text{Nationality}}$ Country	0.011	0.892
MusicalBand (DB111K-174, Inferred)	Single $\xrightarrow{\text{byMusicalArtist}}$ Band	0.444	1.000
	Single $\xrightarrow{\text{Producer}}$ Band	0.107	0.765
	Single $\xrightarrow{\text{Writer}}$ Band	0.140	0.129
	Single $\xrightarrow{\text{Producer}}$ MusicalArtist $\xrightarrow{\text{associatedBand}}$ Band	0.135	0.122
	Single $\xrightarrow{\text{subsequentWork}}$ Single $\xrightarrow{\text{byMusicalArtist}}$ Band	0.126	0.719

selected parameters, we conducted a series of experiments for transductive KG reasoning on DB111K-174 and per-relation link prediction on the YAGO26K-906 (on relation *isCitizenOf*). Our empirical observations indicate that the optimal values for (λ_1, λ_2) for these distinct tasks are (10, 1) and (5, 1), respectively. Moreover, it is noteworthy that performance deteriorates by 12% and 7% respectively when λ_1 is excessively large. This downturn in performance is possibly due to the arrival item, devised to facilitate training, being swallowed up in the reward signal when in these extreme cases. However, interestingly, even when faced with overly large values for (λ_1, λ_2) , the degradation in performance was not significant. This, in turn, underscores that our approach maintains a smooth performance trajectory irrespective of the values of (λ_1, λ_2) , denoting a low sensitivity to these parameters.

D. Inference Time

We pose that our model offers superior efficiency during inference since it mainly search for the target entity type within its corresponding local neighborhood indicated by

the schema graph. This sharply contrasts with the markedly lower efficiency of other meta-path discovery methods, which necessitate exhaustive exploration of the instance graph. The primary computational expense during inference for SchemaWalk rests in the computation of probabilities for all potential outgoing edges along the defined meta-path. Therefore, the inference time of SchemaWalk is fundamentally contingent on the degree distribution of the graph. Assuming the schema graph $\mathcal{G}_s$ conforms to a power law degree distribution, the test-time complexity per epoch, based on the paper's notations, is given as $\mathcal{O}(B'(l-1)\frac{\eta}{\eta-1})$, represents the power law's coefficient. Please refer to Appendix E.2, available online for an in-depth analysis.

VI. CONCLUSION AND FUTURE WORK

In this paper, we investigate how to learn meta-paths in schema-complex HINs. We successfully design a meta-path discovery framework, SchemaWalk, which allows an RL agent to walk on the schema graph, and learn from the rewards defined on the instance graph. SchemaWalk offers several advantages. First,

among all meta-path reasoning methods, SchemaWalk is the pioneering multi-relation reasoning method for HINs, and can fascinatingly, predict fruitful meta-paths for relations without specific evidence. Besides, in contrast to methods of learning meta-paths on the instance graph, SchemaWalk is more effective, as it directly obtains high-quality meta-paths without requiring summarization from partial observations of path instances. Additionally, in comparison to hand-crafted or graph traversal-based methods, SchemaWalk is more efficient at exploring large combinatorial search space and can thus handle extensive schema graphs. Finally, when compared to embedding-based approaches, SchemaWalk demonstrates superior instance-level explainability and generalization ability, particularly when reasoning about unseen entities, as it leverages the mined meta-paths rather than embedding vectors for predictions.

We present the first inductive meta-path learning model, which could be applied to reason multiple relations after a single training. The trained agent could not only learn high-quality meta-paths for a set of given relations, and also output rewarding meta-paths for untrained relations. The query answering and link prediction experiments fully verify the effectiveness of learning meta-paths in this way. The presented results also highlight the importance of meta-paths, not only in schema-simple HINs like Chem2Bio2RDF, but also in schema-complex HINs like KBs, where embedding-based models take dominant roles. Once high-quality meta-paths are obtained for schema-complex HINs, we could achieve better and explainable performances.

Future work includes at least four aspects. First, we could combine SchemaWalk's rewards with the evaluation metrics for downstream tasks and obtain more task-specific meta-paths. Besides, advances in graph representation learning and RL may enable us to design better agent and further improve the performance. Furthermore, we could build a score estimator to reduce the time to evaluate each meta-path during learning. Lastly, we could integrate temporal logic into our reasoning method, thereby making it applicable to temporal HIN.

ACKNOWLEDGMENTS

The authors would like to express our gratitude to Ziniu Hu for his valuable advice. We would also like to thank Jie Kang and Yuehang Si for providing us with some baseline results.

REFERENCES

- [1] S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, and Z. Ives, "DBpedia: A nucleus for a web of open data," in *The Semantic Web*. Berlin, Germany: Springer, 2007, pp. 722–735.
- [2] X. L. Dong, "Challenges and innovations in building a product knowledge graph," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 2869–2869.
- [3] H. M. Berman et al., "The protein data bank," *Nucleic Acids Res.*, vol. 28, no. 1, pp. 235–242, 2000.
- [4] Y. Sun, Y. Yu, and J. Han, "Ranking-based clustering of heterogeneous information networks with star network schema," in *Proc. 15th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2009, pp. 797–806.
- [5] X. Yu et al., "Personalized entity recommendation: A heterogeneous information network approach," in *Proc. 7th ACM Int. Conf. Web Search Data Mining*, 2014, pp. 283–292.
- [6] M. Gönen, "Predicting drug–target interactions from chemical and genomic kernels using Bayesian matrix factorization," *Bioinformatics*, vol. 28, no. 18, pp. 2304–2310, 2012.
- [7] C. Shi, Y. Li, J. Zhang, Y. Sun, and S. Y. Philip, "A survey of heterogeneous information network analysis," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 1, pp. 17–37, Jan. 2017.
- [8] Y. Sun, R. Barber, M. Gupta, C. C. Aggarwal, and J. Han, "Co-author relationship prediction in heterogeneous bibliographic networks," in *Proc. IEEE Int. Conf. Adv. Social Netw. Anal. Mining*, 2011, pp. 121–128.
- [9] X. Yu et al., "Recommendation in heterogeneous information networks with implicit user feedback," in *Proc. 7th ACM Conf. Recommender Syst.*, 2013, pp. 347–350.
- [10] M. Ley, "The DBLP computer science bibliography: Evolution, research issues, perspectives," in *Proc. Int. Symp. String Process. Inf. Retrieval*, Springer, 2002, pp. 1–10.
- [11] T.-Y. Fu, W.-C. Lee, and Z. Lei, "Hin2vec: Explore meta-paths in heterogeneous information networks for representation learning," in *Proc. ACM Conf. Inf. Knowl. Manage.*, 2017, pp. 1797–1806.
- [12] C. Shi, B. Hu, W. X. Zhao, and S. Y. Philip, "Heterogeneous information network embedding for recommendation," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 2, pp. 357–370, Feb. 2019.
- [13] K. Cheng, J. Liu, W. Wang, and Y. Sun, "Rlogic: Recursive logical rule learning from knowledge graphs," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 179–189.
- [14] R. Burke, F. Vahedian, and B. Mobasher, "Hybrid recommendation in heterogeneous networks," in *Proc. Int. Conf. User Model. Adapt. Personalization*, Springer, 2014, pp. 49–60.
- [15] C. Wang et al., "Relsim: Relation similarity search in schema-rich heterogeneous information networks," in *Proc. SIAM Int. Conf. Data Mining*, 2016, pp. 621–629.
- [16] X. Kong, P. S. Yu, Y. Ding, and D. J. Wild, "Meta path-based collective classification in heterogeneous information networks," in *Proc. 21st ACM Int. Conf. Inf. Knowl. Manage.*, 2012, pp. 1567–1571.
- [17] Z. Zhu, R. Cheng, L. Do, Z. Huang, and H. Zhang, "Evaluating top-k meta path queries on large heterogeneous information networks," in *Proc. IEEE Int. Conf. Data Mining*, 2018, pp. 1470–1475.
- [18] C. Yang, M. Liu, F. He, X. Zhang, J. Peng, and J. Han, "Similarity modeling on heterogeneous networks via automatic path discovery," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discov. Databases*, Springer, 2018, pp. 37–54.
- [19] C. Meng, R. Cheng, S. Mani, P. Senellart, and W. Zhang, "Discovering meta-paths in large heterogeneous information networks," in *Proc. 24th Int. Conf. World Wide Web*, 2015, pp. 754–764.
- [20] R. Das et al., "Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning," 2017, *arXiv: 1711.05851*.
- [21] S. Ji, S. Pan, E. Cambria, P. Martinen, and S. Y. Philip, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 2, pp. 494–514, Feb. 2022.
- [22] N. Lao, T. Mitchell, and W. Cohen, "Random walk inference and learning in a large scale knowledge base," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2011, pp. 529–539.
- [23] K. Guu, J. Miller, and P. Liang, "Traversing knowledge graphs in vector space," 2015, *arXiv:1506.01094*.
- [24] K. Toutanova, X. V. Lin, W.-T. Yih, H. Poon, and C. Quirk, "Compositional learning of embeddings for relation paths in knowledge base and text," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics*, 2016, pp. 1434–1444.
- [25] M. Gardner, P. Talukdar, J. Krishnamurthy, and T. Mitchell, "Incorporating vector space similarity in random walk inference over knowledge bases," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2014, pp. 397–406.
- [26] W. Xiong, T. Hoang, and W. Y. Wang, "Deeppath: A reinforcement learning method for knowledge graph reasoning," 2017, *arXiv: 1707.06690*.
- [27] Y. Shen, J. Chen, P.-S. Huang, Y. Guo, and J. Gao, "M-walk: Learning to walk over graphs using Monte Carlo tree search," 2018, *arXiv: 1802.04394*.
- [28] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *Proc. Adv. Neural Inf. Process. Syst.*, 2013, pp. 2787–2795.
- [29] S. He, K. Liu, G. Ji, and J. Zhao, "Learning to represent knowledge graphs with Gaussian embedding," in *Proc. 24th ACM Int. Conf. Inf. Knowl. Manage.*, 2015, pp. 623–632.
- [30] Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang, "Rotate: Knowledge graph embedding by relational rotation in complex space," 2019, *arXiv: 1902.10197*.
- [31] Y. Dong, N. V. Chawla, and A. Swami, "Metapath2vec: Scalable representation learning for heterogeneous networks," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 135–144.

- [32] R. Agrawal, T. Imieliński, and A. Swami, "Mining association rules between sets of items in large databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1993, pp. 207–216.
- [33] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, no. 3, pp. 229–256, 1992.
- [34] R. S. Sutton, D. A. McAllester, S. P. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2000, pp. 1057–1063.
- [35] V. R. Konda and J. N. Tsitsiklis, "Actor-critic algorithms," in *Proc. Adv. Neural Inf. Process. Syst.*, 2000, pp. 1008–1014.
- [36] B. Yang, W.-T. Yih, X. He, J. Gao, and L. Deng, "Embedding entities and relations for learning and inference in knowledge bases," 2014, *arXiv:1412.6575*.
- [37] T. Trouillon, J. Welbl, S. Riedel, É. Gaussier, and G. Bouchard, "Complex embeddings for simple link prediction," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2016, pp. 2071–2080.
- [38] M. Qu, J. Chen, L.-P. Xhonneux, Y. Bengio, and J. Tang, "RNNLogic: Learning logic rules for reasoning on knowledge graphs," 2020, *arXiv:2010.04029*.
- [39] H. Fang, Y. Liu, Y. Cai, and M. Sun, "MLN4KB: An efficient Markov logic network engine for large-scale knowledge bases and structured logic rules," in *Proc. ACM Web Conf.*, 2023, pp. 2423–2432.
- [40] G. Wan, B. Du, S. Pan, and G. Haffari, "Reinforcement learning based meta-path discovery in large-scale heterogeneous information networks," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 6094–6101.
- [41] N. Lao and W. W. Cohen, "Relational retrieval using a combination of path-constrained random walks," *Mach. Learn.*, vol. 81, no. 1, pp. 53–67, 2010.
- [42] J. Zhao, X. Wang, C. Shi, Z. Liu, and Y. Ye, "Network schema preserving heterogeneous information network embedding," in *Proc. Int. Joint Conf. Artif. Intell.*, 2021, pp. 1366–1372.
- [43] F. M. Suchanek, G. Kasneci, and G. Weikum, "Yago: A core of semantic knowledge," in *Proc. 16th Int. Conf. World Wide Web*, 2007, pp. 697–706.
- [44] J. Hao, M. Chen, W. Yu, Y. Sun, and W. Wang, "Universal representation learning of knowledge bases by jointly embedding instances and ontological concepts," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 1709–1719.
- [45] T. Mitchell et al., "Never-ending learning," *Commun. ACM*, vol. 61, no. 5, pp. 103–115, 2018.
- [46] B. Chen et al., "Chem2Bio2RDF: A semantic framework for linking and data mining chemogenomic and systems chemical biology data," *BMC Bioinf.*, vol. 11, no. 1, pp. 1–13, 2010.
- [47] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 701–710.
- [48] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. Van Den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *Proc. 15th Int. Conf. Semantic Web*, Springer, 2018, pp. 593–607.
- [49] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
- [50] X. Wang et al., "Heterogeneous graph attention network," in *Proc. World Wide Web Conf.*, 2019, pp. 2022–2032.
- [51] X. Fu, J. Zhang, Z. Meng, and I. King, "MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding," in *Proc. Web Conf.*, 2020, pp. 2331–2341.
- [52] Z. Hu, Y. Dong, K. Wang, and Y. Sun, "Heterogeneous graph transformer," in *Proc. Web Conf.*, 2020, pp. 2704–2710.
- [53] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 11 106–11 115.
- [54] R. E. Bank and C. C. Douglas, "Sparse matrix multiplication package (SMMP)," *Adv. Comput. Math.*, vol. 1, no. 1, pp. 127–137, 1993.



Shixuan Liu received the BS degree in systems engineering from the National University of Defense Technology, Changsha, China, in 2019, where he is currently working toward the PhD degree. His research interests include reinforcement learning, knowledge reasoning and causal discovery.



a number of refereed journals and conference proceedings, such as *Nature Machine Intelligence*, *Nature Communications*, AAAI, CIKM, etc.



Kewei Cheng received the bachelor's degree from Sichuan University and the master's degree from Arizona State University. She is currently working toward the PhD degree with UCLA. Her research interests are generally in data mining, artificial intelligence, and machine learning, with a focus on neural-symbolic knowledge graph reasoning.



Yunfei Wang received the BS degree in civil engineering from the Hunan University, Changsha, China, in 2020. She is currently working toward the PhD degree with the National University of Defense Technology, Changsha, China. Her research interests include auto penetration test, reinforcement learning, and cyber-security.



several high-level international conferences, workshops, and journals.

Peng Cui (Senior Member, IEEE) received the PhD degree from Tsinghua University in 2010. He is an associate professor with tenure in Tsinghua University. His research interests include causally-regularized machine learning, network representation learning, and social dynamics modeling. He has published more than 100 papers in prestigious conferences and journals in data mining and multimedia. Now, his research is sponsored by National Science Foundation of China, Samsung, Tencent, etc. He also serves as guest editor, co-chair, PC member, and reviewer of



Yizhou Sun (Member, IEEE) received the bachelor's and master's degrees in computer science and statistics from Peking University, China and the PhD degree from the Computer Science Department, University of Illinois at Urbana Champaign (UIUC) in December 2012. She is an associate professor with Computer Science, UCLA. Prior to that, she joined Northeastern University as an assistant professor in 2013.



Zhong Liu received the BS degree in physics from Central China Normal University, Wuhan, Hubei, China, in 1990, the MS degree in computer software and the PhD degree in management science and engineering both from National University of Defense Technology, Changsha, China, in 1997 and 2000. He is a professor with the College of Systems Engineering, National University of Defense Technology, Changsha, China. His research interests include intelligent information systems, and intelligent decision making.